

PROJECT REPORT

Edge-Based Driver Drowsiness and Distraction Alert System

A Real-Time Computer Vision System for Fatigue and Inattention Monitoring

Course Name: Computer Vision

Submitted by: Manav Tiwari

Registration Number: 24BAI10367

GitHub Repository: github.com/Manav-Tiwari/Driver-Drowsiness-Detection

Table of Contents

Section	Title	Page Number
1.	Introduction	3
2.	Problem Statement	3
3.	Functional Requirements	4
4.	Non-Functional Requirements	5
5.	System Architecture	5
6.	Design Diagrams	7
7.	Design Decisions and Rationale	11
8.	Implementation Details	12
9.	Screenshots / Results	13
10.	Testing Approach	14
11.	Challenges Faced	15
12.	Learnings and Key Takeaways	15
13.	Future Enhancements	16
14.	References	16

1. Introduction

Fatigue and inattention behind the wheel remain among the most persistent causes of road accidents. National transport safety studies consistently report that a driver's diminished alertness, whether from drowsiness, momentary distraction, or micro-sleep, contributes to a disproportionately large share of single-vehicle and rear-end collisions. Most commercial fatigue-monitoring products rely on infrared sensors, specialised dashboard hardware, or cloud-based deep learning inference, which raises cost, adds latency, and depends on network connectivity that is not always available inside a moving vehicle.

This project, titled the Edge-Based Driver Drowsiness and Distraction Alert System, was undertaken as the course project for Computer Vision. It applies classical, geometry-based computer vision techniques rather than a heavyweight neural network to a live webcam feed, in order to estimate a driver's eye closure, yawning, and head orientation in real time, entirely on commodity CPU hardware and without any dependency on the internet.

The system is built around Google's MediaPipe Face Mesh model, which returns 468 facial landmark coordinates per frame. From these landmarks, the application derives two well-established geometric ratios, the Eye Aspect Ratio (EAR) and the Mouth Aspect Ratio (MAR), together with a Perspective-n-Point (PnP) based estimate of head pitch, yaw, and roll. These signals are smoothed using a per-metric timer and passed through a small evaluator that decides when a genuine drowsiness, yawning, or distraction event has occurred, at which point the driver is alerted through a non-blocking audio cue and the event is written to a local database for later review.

2. Problem Statement

2.1 Background

Driver fatigue develops gradually and is difficult for drivers to self-assess. By the time a driver consciously feels sleepy, reaction time has often already degraded significantly. Existing mitigation approaches fall into two broad categories: specialised in-cabin hardware such as infrared eye-trackers fitted by vehicle manufacturers, which are expensive and not easily retrofitted to older vehicles, and smartphone or cloud-based applications that stream video for server-side deep learning inference, which introduces latency, privacy concerns around continuous video transmission, and a hard dependency on network availability that cannot be guaranteed on highways or in remote areas.

2.2 Problem Definition

There is a need for a lightweight, offline-first software system that can run on ordinary consumer hardware, such as a laptop connected to a standard webcam, and reliably distinguish between an alert driver, a drowsy driver, a yawning driver, and a visually distracted driver, all in real time and without transmitting raw video to any external server.

2.3 Proposed Solution

This project proposes a geometry-driven computer vision pipeline that extracts dense facial landmarks using MediaPipe's Face Mesh model, computes interpretable numerical indicators (EAR, MAR, head pitch and yaw) from those landmarks, and evaluates them against calibrated thresholds using per-metric timers to suppress false positives caused by normal blinking or momentary glances. When a sustained anomaly is detected, the system

raises an audible alert and persists the incident to a local SQLite database, giving the driver, or a fleet operator reviewing the log after a trip, a complete, privacy-preserving record of fatigue and distraction events.

3. Functional Requirements

The system is structured into three primary functional modules: Data Acquisition and Landmark Extraction, Metric Computation and Drowsiness Evaluation, and Alerting and Incident Logging.

3.1 Module 1: Data Acquisition and Landmark Extraction

- FR-1: The system shall capture a continuous video stream from a connected webcam through the CameraStream class, at a configurable resolution, camera index, and target frame rate (defaults: 640 by 480 at 30 FPS).
- FR-2: The system shall process each captured frame through the FaceLandmarkExtractor class, which wraps MediaPipe Face Mesh, to extract 468 facial landmark coordinates in both 2D pixel space and normalised 3D space.
- FR-3: The system shall gracefully handle frames in which no face is detected, for example due to occlusion, by displaying a "Face Not Found" status rather than terminating the video loop.

3.2 Module 2: Metric Computation and Drowsiness Evaluation

- FR-4: The system shall compute the Eye Aspect Ratio (EAR) for both eyes from the relevant landmark subset on every processed frame, via compute_ear().
- FR-5: The system shall compute the Mouth Aspect Ratio (MAR) to identify yawning behaviour, via compute_mar().
- FR-6: The system shall estimate head pitch, yaw, and roll using Perspective-n-Point (PnP) pose estimation, via compute_head_pose(), to identify whether the driver is looking away from the road.
- FR-7: The system shall apply temporal smoothing so that an alert is raised only when a threshold is exceeded continuously for a configurable duration, rather than on a single noisy frame. The StateEvaluator class implements this using independent start-time timers for EAR, MAR, and head pose.
- FR-8: The system shall classify the driver's state into distinct categories: Awake, Drowsiness Detected, Yawning Detected, and Distraction Detected.

3.3 Module 3: Alerting and Incident Logging

- FR-9: The system shall issue a non-blocking audio alert through the AudioAlerter class using a background thread, so that alert playback never stalls the video processing loop.
- FR-10: The system shall record every confirmed incident, including a timestamp, the event type, the triggering metric value, and the duration for which the metric stayed past its threshold, to a local SQLite database (incidents.db) through the IncidentLogger class.
- FR-11: The system shall display the live video feed overlaid with the computed EAR, MAR, pitch, and yaw values and the current driver state, for real-time visual feedback.

3.4 Input / Output Summary

Module	Input	Output
Data Acquisition and Landmark Extraction	Live webcam frame (BGR image)	468 facial landmark coordinates in 2D and 3D
Metric Computation and Evaluation	Landmark coordinates and configured thresholds	EAR, MAR, head pose angles, and a driver-state label
Alerting and Logging	Driver-state label and metric values	Audio alert (if triggered) and a persisted incident record

4. Non-Functional Requirements

Beyond the functional feature set, the following non-functional requirements were defined to guide implementation decisions:

- **Performance:** The end-to-end pipeline (capture, landmark extraction, metric computation, evaluation) must sustain more than 15 frames per second on a standard laptop CPU with no dedicated GPU, so that drowsiness onset is detected with minimal latency.
- **Reliability:** The system must not crash or hang when the face is temporarily occluded, when lighting conditions change abruptly, or when the webcam briefly fails to return a frame. `CameraStream.read_frame()` reports failure explicitly rather than raising an unhandled exception.
- **Resource Efficiency:** Camera handles, the MediaPipe Face Mesh context, and the Pygame audio mixer are released deterministically on shutdown (via `release()` methods and Python context managers) to avoid handle or memory leaks during long monitoring sessions.
- **Usability:** On-screen overlays (state label, EAR, MAR, pitch, and yaw values) must be legible at a glance, so a driver can understand system status without diverting attention from the road for more than a fraction of a second.
- **Security and Privacy:** No raw video frames are transmitted off-device or persisted to disk; only derived numerical incident metadata is stored locally in `incidents.db`, minimising privacy exposure.
- **Maintainability:** The codebase is split into single-responsibility modules (camera, landmarks, metrics, evaluator, alerter, logger, config) so that any one concern, for example replacing MediaPipe with another landmark model, can be modified in isolation.
- **Error Handling and Logging:** Camera access, database writes, and audio playback are each isolated in their own class so that a failure in one subsystem can be reported without silently propagating into a crash.
- **Scalability:** The modular structure allows the same evaluator and logging pipeline to later be extended to multiple simultaneous camera feeds, for example a fleet-management use case, with limited rework.

5. System Architecture

The system follows a layered pipeline architecture composed of three conceptual layers, Acquisition, Processing/Inference, and Response, coordinated by a central orchestrator, `main.py`. Each layer is implemented as one or more independent Python modules with a narrow, well-defined interface, which keeps the pipeline testable and easy to reason about.

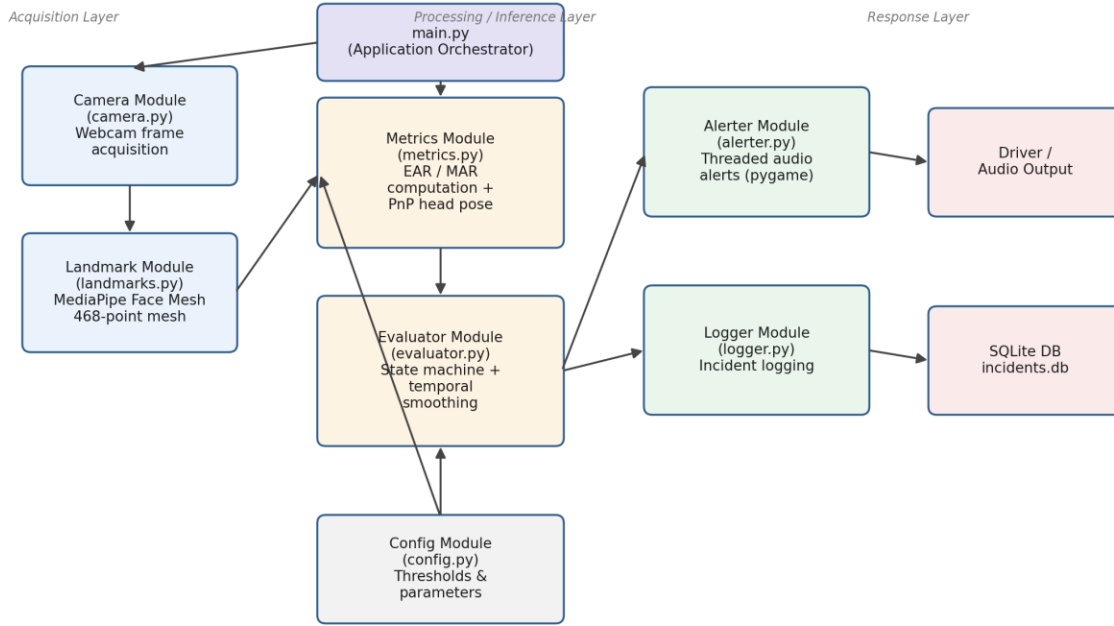


Figure 1: High-level system architecture of the Driver Drowsiness and Distraction Alert System.

Frames flow from CameraStream into FaceLandmarkExtractor, which invokes MediaPipe Face Mesh to obtain the 468-point mesh. The metrics module converts these raw coordinates into interpretable signals (EAR, MAR, head pose), which StateEvaluator compares against thresholds defined in config.py, tracking a separate start-time for each metric so that an alert only fires once a metric has remained past its threshold continuously for a configured duration. When StateEvaluator confirms a sustained event, it calls AudioAlerter to play an audio cue on a background thread and IncidentLogger to persist the incident to the local SQLite database, subject to a five-second cooldown between log writes for the same evaluator instance.

6. Design Diagrams

6.1 Use Case Diagram

Two actors interact with the system: the Driver, who is monitored in real time and receives audio alerts, and a Fleet Administrator, who may review logged incidents after a trip or adjust detection thresholds for a specific vehicle or driver profile.

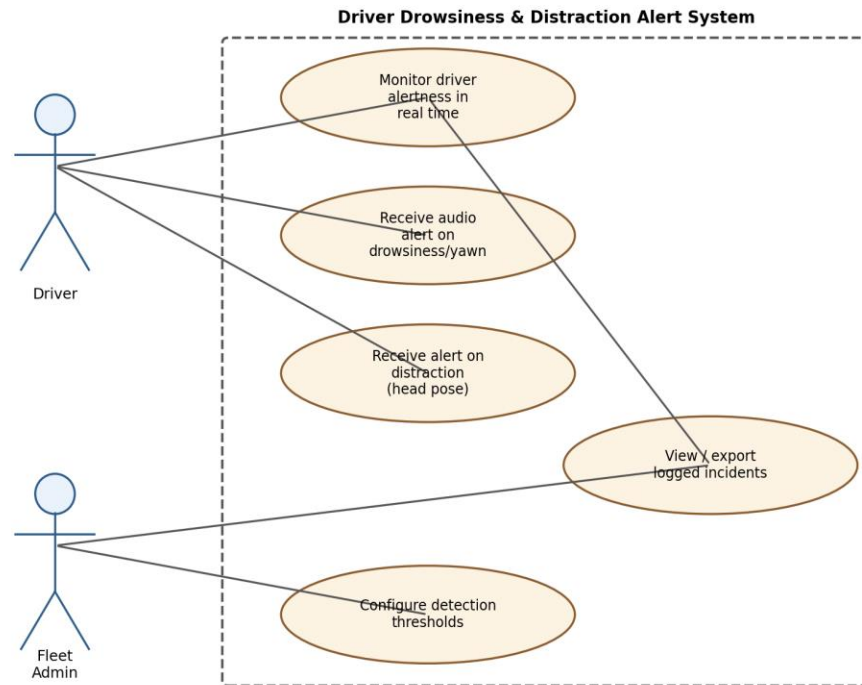


Figure 2: Use case diagram.

6.2 Process Flow / Workflow Diagram

The workflow diagram below traces one iteration of the main monitoring loop, from frame capture through metric computation to the alerting decision, and shows how the loop continues indefinitely while the application is running.

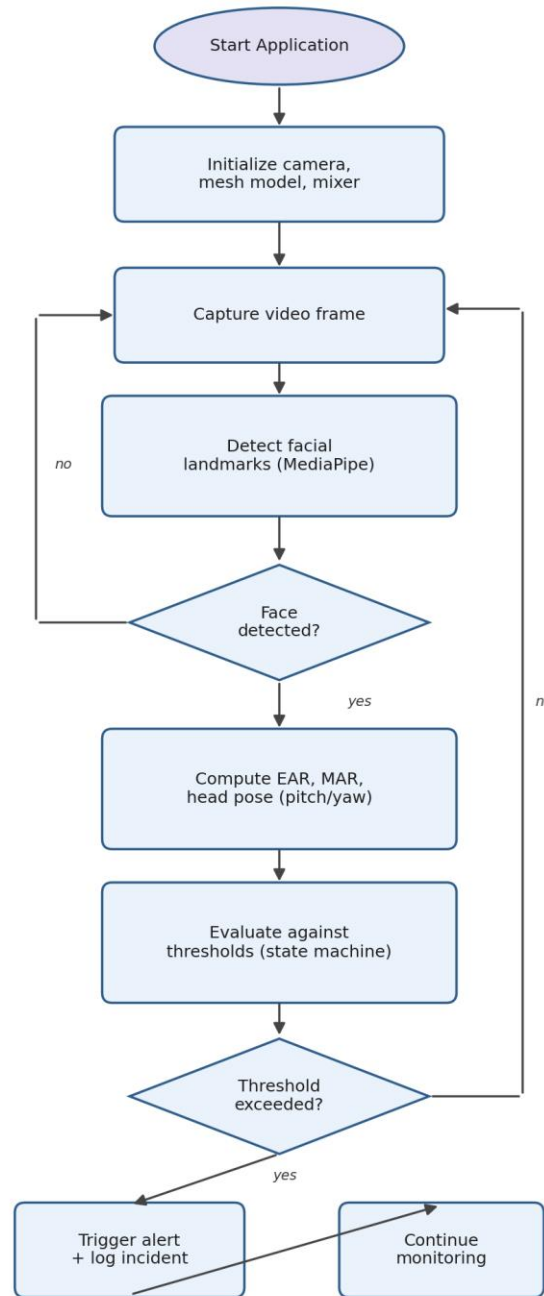


Figure 3: Process flow / workflow diagram of the monitoring loop.

6.3 Sequence Diagram

The sequence diagram illustrates the runtime message flow for a single processed frame, showing how `main()` coordinates `CameraStream`, `FaceLandmarkExtractor`, the metrics functions, `StateEvaluator`, `AudioAlerter`, and `IncidentLogger`.

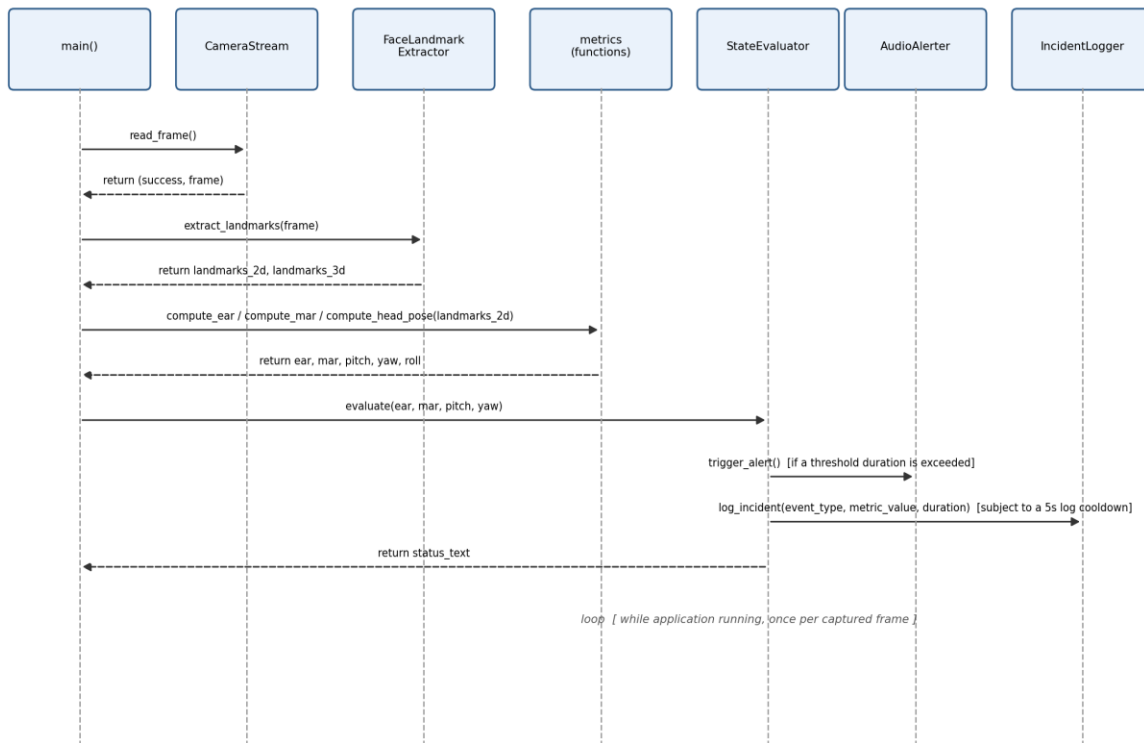


Figure 4: Sequence diagram for one monitoring cycle.

6.4 Class / Component Diagram

The class diagram maps each conceptual responsibility onto the corresponding Python class or module in the repository, showing the principal attributes, methods, and dependency directions between components. `metrics.py` and `config.py` are implemented as plain modules of functions and constants rather than classes, which is reflected in the diagram.

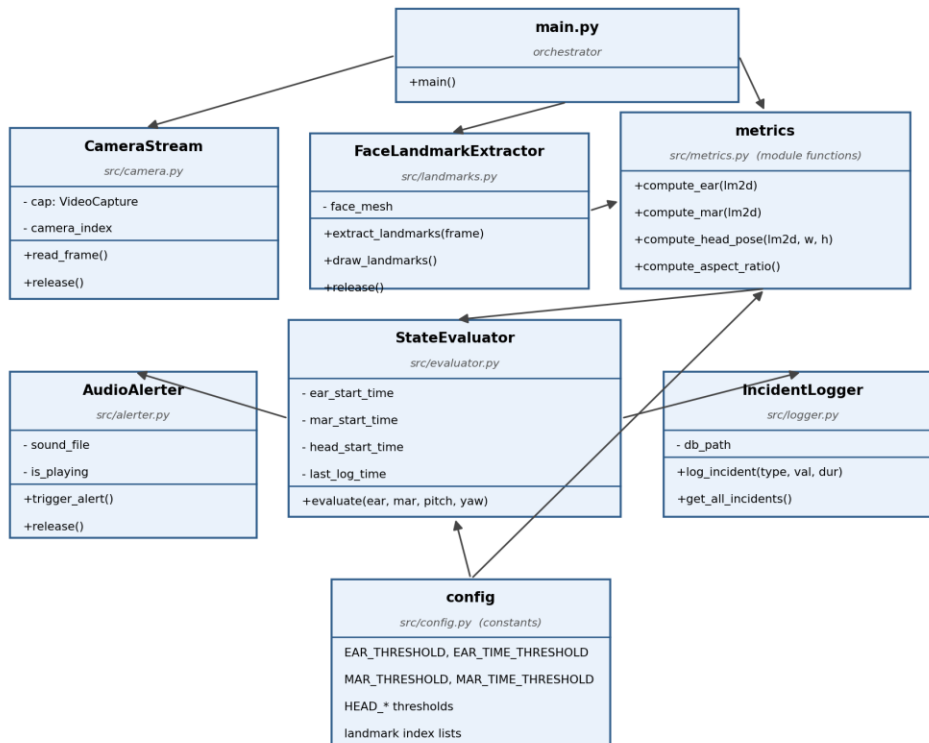


Figure 5: Class / component diagram.

6.5 Database Schema / ER Diagram

The only persistent store in the system is a single-table SQLite database used for incident logging. Its schema is intentionally denormalised, since incidents are independent, append-only records with no relational structure to other entities.

incidents		
id : INTEGER, AUTOINCREMENT		PK
timestamp : TEXT (ISO-8601, via datetime.now())		
event_type : TEXT ('DROWSINESS' 'YAWNING' 'DISTRACTION')		
metric_value : REAL (the EAR, MAR, or yaw value that triggered the event)		
duration : REAL (seconds the metric stayed past threshold)		

Figure 6: Schema of the incidents table in incidents.db, as created by IncidentLogger._init_db().

7. Design Decisions and Rationale

7.1 MediaPipe Face Mesh over Dlib or a Deep CNN Landmark Model

Dlib's classical HOG-based facial landmark detector and full convolutional-network face-detection pipelines were both considered. MediaPipe Face Mesh was ultimately selected because it is built on BlazeFace, an architecture specifically optimised for mobile and edge CPUs, giving it a substantially smaller inference footprint than a general-purpose CNN while still returning a denser 468-point mesh than Dlib's 68-point model, which in turn improves the precision of the eye and mouth aspect ratio calculations. `refine_landmarks` is enabled in `FaceLandmarkExtractor` for improved lip and eye point accuracy.

7.2 Geometric Ratios (EAR/MAR) over a Learned Classifier

An alternative design would train a binary eyes-open or eyes-closed classifier on a labelled image dataset. This project instead computes EAR and MAR directly from landmark geometry, using a single generic `compute_aspect_ratio()` helper shared by both metrics. The geometric approach requires no training data or model retraining across different faces, lighting conditions, or camera angles, is fully interpretable, and executes as a handful of Euclidean distance calculations rather than a forward pass through a network, directly supporting the performance and resource-efficiency non-functional requirements.

7.3 Temporal Smoothing via Per-Metric Timers

A single frame in which EAR dips below the threshold is not evidence of drowsiness; it is far more often a normal blink, which lasts only a few hundred milliseconds. `StateEvaluator` therefore keeps a separate start-time for EAR, MAR, and head pose, and only raises an alert once the elapsed duration since that start-time exceeds a calibrated minimum (2.5 seconds for EAR, 2.0 seconds for MAR, and 3.0 seconds for head pose, as set in `config.py`). This design choice directly reduces false-positive alerts without materially delaying true-positive detection.

7.4 SQLite for Incident Logging

SQLite was chosen over a client-server database such as PostgreSQL or MySQL because the system must operate fully offline inside a vehicle with no guaranteed network connectivity. SQLite requires no separate server process, stores the entire database as a single portable file, and is included in Python's standard library, which keeps the deployment footprint minimal. A five-second cooldown in `StateEvaluator` prevents the same sustained event from writing duplicate rows on every frame.

7.5 Threaded, Self-Contained Audio Alerts

Audio playback via Pygame's mixer is dispatched on a background thread rather than the main video-processing thread. If alert playback were synchronous, a multi-second audio clip could stall frame capture and landmark extraction, directly degrading the real-time performance the non-functional requirements demand. `AudioAlerter` also removes an external dependency on a supplied sound file: if `assets/alert.wav` is not already present, it generates a simple 440 Hz tone with NumPy and SciPy the first time the application runs, keeping the project fully self-contained.

8. Implementation Details

8.1 Technology Stack

Component	Technology
Programming Language	Python 3.x
Video Capture and Pose Geometry	OpenCV
Facial Landmark Inference	MediaPipe Face Mesh
Numerical / Geometric Computation	NumPy, SciPy
Incident Persistence	SQLite3 (Python standard library)
Audio Alerting	Pygame (threaded mixer)
Testing	Pytest
Version Control	Git / GitHub

8.2 Module Breakdown

The implementation is organised into the following files, matching the architecture and class diagrams presented earlier:

- `main.py`: application entry point; owns the primary capture-process-evaluate loop and wires all other modules together using Python context managers for `CameraStream` and `FaceLandmarkExtractor`.
- `src/camera.py` (`CameraStream`): thin wrapper around OpenCV's `VideoCapture`, responsible for frame acquisition, resolution/FPS configuration, and camera lifecycle management.
- `src/landmarks.py` (`FaceLandmarkExtractor`): wraps the MediaPipe Face Mesh model and exposes `extract_landmarks(frame)`, returning both 2D pixel and normalised 3D landmark coordinates, plus a `draw_landmarks()` utility for debugging.
- `src/metrics.py`: pure functions for Eye Aspect Ratio, Mouth Aspect Ratio, and PnP-based head pose estimation, built on a shared `compute_aspect_ratio()` helper.
- `src/evaluator.py` (`StateEvaluator`): tracks a start-time per metric and decides the driver's current state once a threshold has been exceeded continuously for long enough; also applies a five-second cooldown before logging a repeat incident.
- `src/alserter.py` (`AudioAlerter`): threaded audio-alert dispatch using Pygame, including on-the-fly generation of a fallback alert tone.
- `src/logger.py` (`IncidentLogger`): SQLite interface that creates the incidents table if it does not exist, inserts new incident rows, and can retrieve all logged incidents.
- `src/config.py`: centralised thresholds and tunable parameters (EAR/MAR cut-offs and time windows, head pose angle limits, camera settings, and MediaPipe landmark index lists), so that calibration never requires touching business logic.
- `tests/`: unit tests covering the aspect-ratio computation and the evaluator's temporal-smoothing behaviour.

8.3 Core Algorithmic Logic

Eye Aspect Ratio and Mouth Aspect Ratio are both computed by the same generic formula: given six ordered landmark points, the ratio is $(|p_2 - p_6| + |p_3 - p_5|)$ divided by $(2 \times |p_1 - p_4|)$, where p_1 and p_4 are the horizontal extremes and the remaining points bracket the vertical opening. This ratio drops toward zero as the eyes close

and rises as the mouth opens during a yawn. Head pose is estimated by solving the Perspective-n-Point problem between six fixed 3D reference points on a generic face model (nose tip, chin, eye corners, and mouth corners) and their corresponding 2D landmark positions using `cv2.solvePnP`, then converting the resulting rotation matrix into pitch, yaw, and roll with `cv2.RQDecomp3x3`.

On each processed frame, `main()` requests a frame from `CameraStream`, passes it to `FaceLandmarkExtractor`, feeds the returned 2D landmarks into the metrics functions, and forwards the resulting EAR, MAR, pitch, and yaw values to `StateEvaluator.evaluate()`. `StateEvaluator` updates its internal timers and, on a confirmed state transition, calls `AudioAlerter.trigger_alert()` and `IncidentLogger.log_incident()` before the loop repeats for the next frame.

8.4 Setup and Execution

The project is installed and run with the following commands, as documented in the repository README:

- Install dependencies: `pip install -r requirements.txt`
- Run the application: `python main.py` (press q inside the video window to exit)
- Run the unit test suite: `pytest tests/`

9. Screenshots / Results

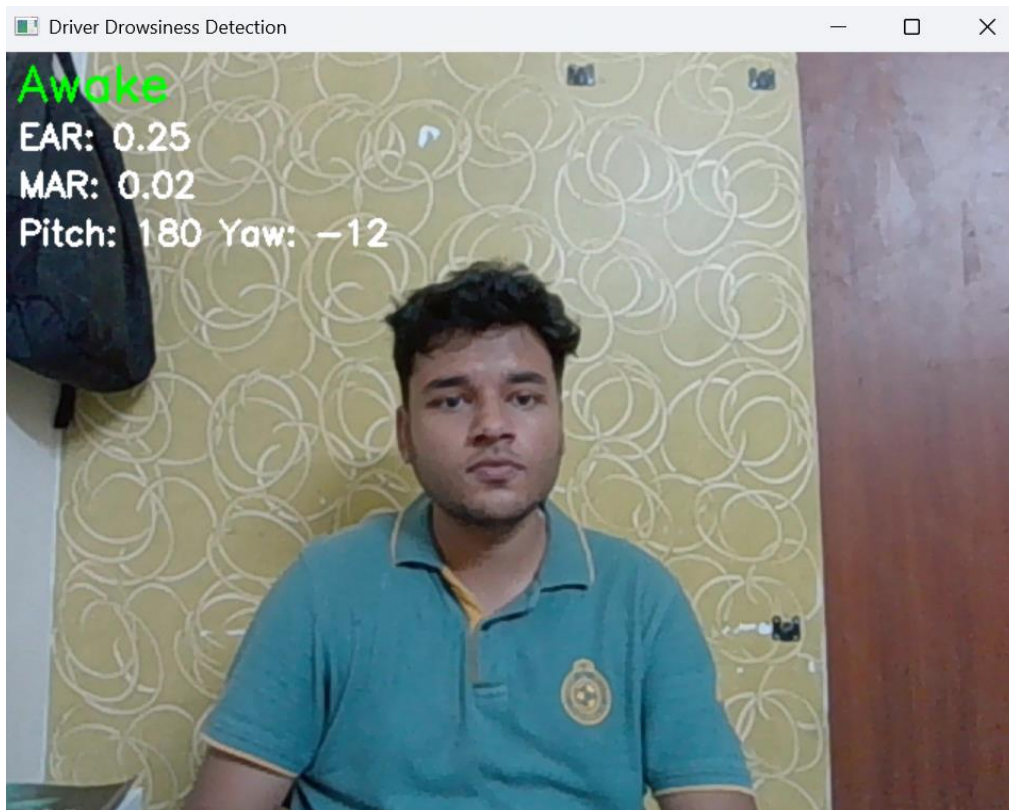


Figure 7: Live webcam overlay demonstrating the alert "Awake" state with normal EAR and MAR values.

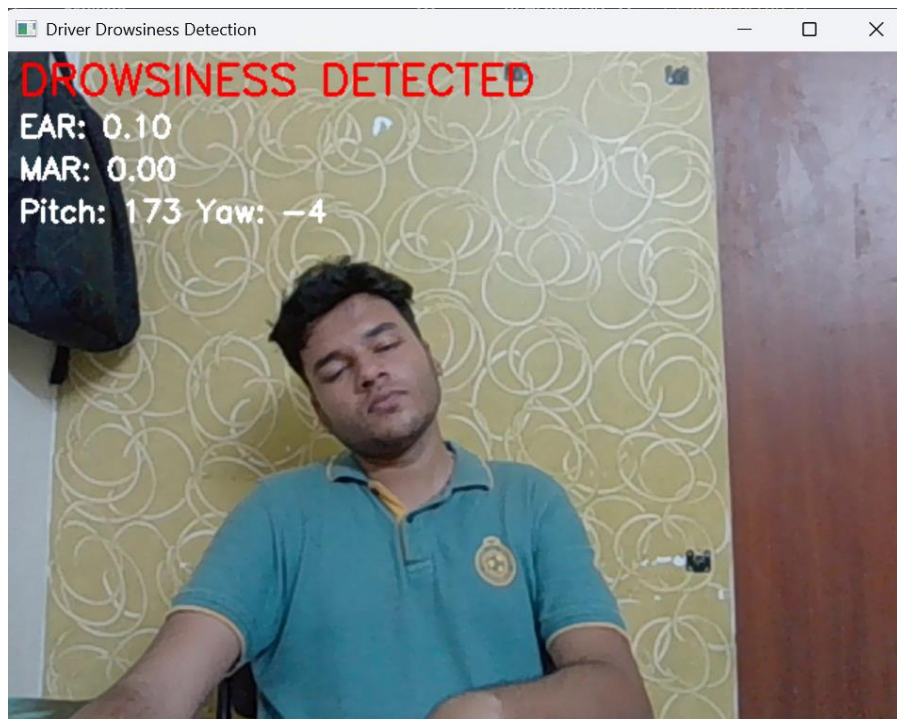


Figure 8: Sustained eye closure triggering the "DROWSINESS DETECTED" alert threshold.

10. Testing Approach

Testing focused on the two areas of the codebase where correctness is both critical and independently verifiable without a live camera: the geometric metric functions and the evaluator's temporal-smoothing logic. The automated tests are in `tests/test_metrics.py` and `tests/test_evaluator.py`.

10.1 Unit Testing

- `test_compute_aspect_ratio()` feeds `compute_aspect_ratio()` a hand-constructed set of six points forming a symmetric hexagon (a mock eye shape) with known distances, and asserts the returned ratio equals the expected value of 0.6667, verifying the core formula shared by EAR and MAR.
- `test_compute_ear_none()` and `test_compute_mar_none()` assert that `compute_ear(None)` and `compute_mar(None)` both return 0.0, confirming the functions fail safely when no face/landmarks are available rather than raising an exception.
- `test_evaluator_drowsiness()` uses lightweight mock `AudioAlerter` and `IncidentLogger` objects to test `StateEvaluator` in isolation. It first calls `evaluate()` with a normal EAR of 0.3 and checks the status is `Awake` and no alert fires. It then calls `evaluate()` with a low EAR of 0.2 and confirms the state does not change immediately, since the time window has not yet elapsed. Finally, it manually rewinds `ear_start_time` by 3 seconds to simulate sustained eye closure and confirms that `evaluate()` now returns `DROWSINESS DETECTED`, that the mock alerter was triggered, and that the mock logger recorded a `DROWSINESS` entry.

10.2 Manual / Integration Testing

- End-to-end manual testing was performed using a live webcam, covering normal alert behaviour, deliberate slow blinking, deliberate yawning, and deliberate head turns, to confirm each alert path fires correctly and that false positives from normal blinking are suppressed.

- Frame rate was monitored during manual testing to confirm the system sustains the target of more than 15 FPS on the development machine's CPU.
- Resource cleanup was verified by running repeated start and stop cycles and confirming that camera and audio-mixer handles are released without leaking, using the `__enter__/__exit__` context-manager methods on `CameraStream` and `FaceLandmarkExtractor`.

10.3 Test Execution

All automated unit tests are executed with the command `pytest tests/`, and are intended to be run before every commit to catch regressions in the core metric and evaluation logic.

11. Challenges Faced

- Threshold calibration: EAR and MAR values vary meaningfully across individuals due to differences in eye shape and camera angle, so a single fixed threshold produced inconsistent results across test subjects. This was mitigated by centralising all thresholds in `config.py` so they can be recalibrated per user without touching detection logic.
- False positives from natural blinking: an early version of the evaluator triggered false drowsiness alerts on ordinary blinks. Introducing per-metric start-time tracking with a minimum-duration requirement in `StateEvaluator` resolved this.
- Audio blocking the main loop: initially, alert playback was synchronous and visibly reduced the frame rate whenever an alert fired. Moving playback to a background thread inside `AudioAlerter` resolved the stutter.
- Duplicate log entries: because `evaluate()` runs on every frame, a single sustained incident could otherwise be written to the database dozens of times per second. Adding the five-second `log_cooldown` inside `StateEvaluator`'s `_log_incident()` helper fixed this during the later debugging pass on the repository.
- Lighting sensitivity: landmark detection quality degraded in low-light or strongly backlit conditions, occasionally causing temporary loss of tracking. The system fails gracefully in these cases by returning `None` landmarks and showing a `Face Not Found` status rather than crashing or emitting spurious alerts.
- Head pose estimation accuracy: because PnP pose estimation depends on an assumed generic 3D facial geometry, it is sensitive to camera field of view and mounting position. Some manual tuning of the head pose angle thresholds (`HEAD_PITCH_THRESHOLD_DOWN`, `HEAD_YAW_THRESHOLD_LEFT/RIGHT`) was needed to obtain stable and meaningfully triggered distraction alerts.

12. Learnings and Key Takeaways

- Classical geometric computer vision techniques, such as landmark-based ratios and PnP pose estimation, can achieve robust, real-time results on constrained hardware without the computational cost of training and deploying a deep learning classifier.
- Temporal reasoning is often as important as the underlying per-frame signal. A naive frame-by-frame threshold check is insufficient for any system observing a continuous, noisy physical process such as human eye movement.
- Designing for offline, edge-first operation from the start (local SQLite storage, no network calls, CPU-only inference) simplifies the system considerably compared to retrofitting privacy and connectivity constraints onto a cloud-first design.
- Separating configuration (thresholds, parameters) from logic (the evaluator, metric computation) made iterative calibration during testing and debugging dramatically faster, reinforcing the value of the single-responsibility module structure adopted for this project.

- Concurrency considerations, such as threaded audio playback and a log-write cooldown, matter even in a fundamentally single-purpose desktop application whenever repeated or blocking I/O could otherwise degrade a time-sensitive processing loop.

13. Future Enhancements

- Per-driver calibration profile: automatically learn a driver's baseline EAR and MAR over the first few minutes of a trip and adjust thresholds accordingly, rather than relying solely on the fixed global constants in `config.py`.
- Infrared or low-light camera support: extend `CameraStream` to support infrared webcams for reliable night-time operation, since visible-light cameras degrade significantly after dark.
- Mobile or embedded deployment: port the pipeline to a Raspberry Pi or equivalent low-power embedded board so the system can be permanently mounted in a vehicle rather than run from a laptop.
- Fleet dashboard: build a lightweight web dashboard that periodically syncs the local SQLite incident log to a central server for fleet-wide reporting, while preserving the offline-first design of the in-vehicle component.
- Additional distraction signals: incorporate phone-usage detection or gaze-direction estimation as further indicators of distraction alongside head pose.
- Haptic feedback channel: add a vibration-based alert output, for example via a connected seat or steering-wheel actuator, as an alternative or supplement to the audio alert for noisy cabin environments.

14. References

- Google AI Edge / MediaPipe documentation, Face Landmarker (Face Mesh) solution: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker
- OpenCV documentation, camera capture and solvePnP-based pose estimation: <https://docs.opencv.org/>
- Soukupova, T. and Cech, J., "Real-Time Eye Blink Detection using Facial Landmarks," 21st Computer Vision Winter Workshop, 2016. Origin of the Eye Aspect Ratio (EAR) metric.
- Python Software Foundation, sqlite3 standard library module documentation: <https://docs.python.org/3/library/sqlite3.html>
- Pygame documentation, mixer module: <https://www.pygame.org/docs/ref/mixer.html>